Strings and String Handling

Strings

- A string is a *sequence* of characters (letters, digits, symbols, spaces) treated as a single piece of data
- In Python, strings are written between quotes
  - single `' ... '`,
  - double `" ... "`, or
  - triple `''' ... '''` / `""" ... """` for multi-line text

Strings are the data type used for any text-based information: names, sentences, file contents, user input, etc.

```
1  name = "Ada"
2  greeting = 'Hello there'
3  paragraph = """This spans
4  multiple lines."""
```

Strings, how are they different

Numbers (`int` , `float`) represent quantities you can do *arithmetic* on

Strings represent *text*, even if that text looks like a number

- `"5" + "3"` does not give `8` , it concatenates to `"53"` , because both operands are strings

Strings are sequences:

- each character has a position (index), so they can be indexed and sliced like a list

Strings are immutable

- once created, a string's characters cannot be changed in place
- any "modification" creates a new string

```
age = 21          # int — can do maths: age + 1 → 22
age_text = "21"    # str — age_text + 1 → TypeError
age_text[0]        # '2' — indexing works
```

The input function

`input()` is a built-in Python function used to get information typed by the user while the program is running

- It pauses the program and waits until the user types something and presses Enter

You can pass an *optional* string as a prompt, which is displayed before the user types

- `input()` always returns a string, even if the user types a number
- if you need a number, you must convert it with `int()` or `float()`

```
1  name = input("What is your name? ")
2  print("Hello, " + name)
3
4  age_text = input("Enter your age: ")
5  age = int(age_text)     # convert string to int
```

Concatenation and Comparisons on Strings

Concatenation joins strings end-to-end using the `+` operator;

- it works because `+` on strings is defined to mean "combine the sequences," not "add numbers"

Only strings can be concatenated with `+`

- mixing types (e.g. `str + int`) raises a `TypeError` unless you convert first

Comparison operators (`==` , `≠` , `<` , `>` , `⩽` , `⩾`) compare strings *character by character* using their underlying character codes (Unicode/ASCII values)

- This means comparisons are case-sensitive and follow alphabetical-ish ordering:
- uppercase letters come before lowercase letters in ASCII

Concatenation and Comparisons on Strings

Examples

```
# Concatenation
first = "Comp"
second = "102"
course = first + second        # "Comp102"
course2 = first + " " + second # "Comp 102"

# Comparisons
"apple" == "apple"    # True
"apple" == "Apple"    # False (case-sensitive)
"apple" < "banana"    # True  (a comes before b)
"Zebra" < "apple"     # True  (uppercase Z < lowercase a in ASCII)
"10" < "9"            # True  (compares character by character, not numerically!)
```

f-strings and string formatting

f-strings (formatted string literals) let you embed expressions directly inside a string, prefixed with `f`

- Anything inside `{ }` is *evaluated* and inserted into the string automatically
- no need for manual concatenation or type conversion

Introduced in Python `3.6+`; the modern, readable way to build strings from variables

```
name = "Ada"
age = 21
print(f"Hello, {name}! You are {age} years old.")
# Hello, Ada! You are 21 years old.

# Expressions work too
print(f"Next year you'll be {age + 1}.")

# Formatting numbers: width, decimal places
pi = 3.14159
print(f"Pi rounded: {pi:.2f}")   # Pi rounded: 3.14
```

Older formatting methods

```
"Hello, {}! You are {} years old.".format(name, age)   # .format()
"Hello, %s! You are %d years old." % (name, age)       # % operator (legacy)
```

Exercise: predict the output

For each line, predict the output *before* running it, then explain why

```
print("5" + "3")
print("5" + 3)
print(int("5") + 3)
print("py" + "thon")
print("ab" * 3)
```

Exercise: true or false

For each, say True or False, and explain your reasoning

1. `"apple" == "Apple"`
2. `"Zebra" < "apple"`
3. `"10" < "9"`
4. `"10" < 9`
5. `len("Hello") == 5`
6. `"cat" + "dog" == "catdog"`

Exercise: fix the bug

Each snippet below crashes or misbehaves. Find the bug and fix it.

```
age = input("Enter your age: ")
next_year = age + 1
print("Next year you'll be", next_year)
```

```
first = "Comp"
second = 102
course = first + second
```

```
score = "87"
print(f"Your score out of 100 is {score * 100}")
```

Exercise: convert to f-strings

Rewrite each of these using an f-string

```
name = "Ada"
score = 91.5

print("Hello, " + name + "! Your score is " + str(score) + ".")

print("{} scored {:.1f} points".format(name, score))

print("%s scored %.1f points" % (name, score))
```

Exercise: short answer

1. Why does `"10" < "9"` evaluate to `True`, even though `10` is bigger than `9`?
2. Why does `input()` always return a string, even when the user types digits?
3. Give one reason strings are described as immutable. What does that actually mean in practice?
4. What is the difference between `"5" + "3"` and `5 + 3`?
5. Name one advantage of f-strings over the `+` concatenation approach.